Java Notes - Method Overloading

Why Do We Use Method Overloading?

• Perform similar tasks using the same method name.
• Improves code readability and reusability.
• Java automatically selects the correct method based on the arguments passed.

Rules of Method Overloading

✓ Same method name.
✓ Different number of parameters.
✓ Different parameter types.
✓ Different order of parameters.
✗ Same method name + same parameters + different return type = Not Allowed.

Different Ways to Overload a Method

1. Changing Number of Parameters
add(int a, int b)
add(int a, int b, int c)

2. Changing Data Types
show(int a)
show(String name)

3. Changing Order of Parameters
display(int a, String name)
display(String name, int a)

Invalid Method Overloading

int add(int a, int b)
double add(int a, int b) // Compile-time Error

Reason: Only the return type is different. The method name and parameters are the same.

Polymorphism

Method Overloading is also known as:
• Compile-Time Polymorphism
• Static Polymorphism
• Early Binding

Example

```
public class Demo {
  static void show(int a){ System.out.println(a); }
  static void show(String name){ System.out.println(name); }
  public static void main(String[] args){
    show(10);
    show("Ritwik");
  }
}
```

Output:

Advantages

• Code reusability
• Better readability
• Easy to understand
• Reduces the number of method names

Quick Revision

Method Overloading = Same method name, different parameters.
Change number, type, or order of parameters.
Return type alone cannot overload a method.
Also called Compile-Time Polymorphism, Static Polymorphism, or Early Binding.